

MAKALAH
IMPLEMENTASI TOOLS JENKINS UNTUK PROSES CI/CD
BERBASIS WINDOWS

Disusun untuk memenuhi tugas Software Quality Assurance yang diampu oleh:
Bapak Karpen, M.Kom



DISUSUN OLEH:
NAZHIF RABBANI
2310031802043

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS SAINS DAN TEKNOLOGI INDONESIA
PEKANBARU
2026

ABSTRAK

Pengembangan perangkat lunak modern menuntut proses integrasi dan pengiriman kode yang cepat, andal, serta efisien. Continuous Integration dan Continuous Deployment (CI/CD) merupakan metodologi DevOps yang memungkinkan otomatisasi seluruh tahapan pengembangan, mulai dari penulisan kode hingga penerapan ke lingkungan produksi. Jenkins adalah salah satu tools open-source paling populer yang digunakan untuk mengimplementasikan pipeline CI/CD. Makalah ini membahas konsep dasar Jenkins, arsitektur sistem, serta tahapan instalasi dan konfigurasi Jenkins di sistem operasi Windows secara mendetail. Pembahasan mencakup prasyarat sistem, instalasi Java Development Kit (JDK), konfigurasi awal Jenkins melalui antarmuka web, pengelolaan plugin, serta pembuatan pipeline CI/CD menggunakan Jenkinsfile deklaratif. Setiap tahapan dijelaskan secara komprehensif disertai gambar ilustrasi dan tabel pendukung. Hasil kajian menunjukkan bahwa Jenkins mampu mengotomatisasi proses build, test, dan deployment secara signifikan, sehingga meningkatkan efisiensi dan kualitas perangkat lunak yang dihasilkan.

Kata Kunci: Jenkins, CI/CD, DevOps, Continuous Integration, Continuous Deployment, Windows, Pipeline, Otomasi

DAFTAR ISI

ABSTRAK.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	5
1.1 Latar Belakang	5
1.2 Rumusan Masalah.....	5
1.3 Tujuan Penulisan.....	6
1.4 Manfaat Penulisan.....	6
BAB II LANDASAN TEORI.....	7
2.1 DevOps	7
2.2 Continuous Integration dan Continuous Deployment (CI/CD)	7
2.3 Jenkins sebagai Automation Server	8
2.4 Arsitektur Jenkins	9
BAB III INSTALASI JENKINS DI WINDOWS	11
3.1 Prasyarat Sistem.....	11
3.2 Instalasi Java Development Kit (JDK)	11
3.3 Instalasi Jenkins melalui Windows Installer.....	13
3.4 Konfigurasi Awal Jenkins.....	17
3.4.1 Membuka Kunci Jenkins (Unlock Jenkins)	17
3.4.2 Pemilihan Plugin Awal	18
3.4.3 Pembuatan Akun Administrator	18
3.5 Manajemen Plugin Jenkins	19
BAB IV IMPLEMENTASI CI/CD PIPELINE DENGAN JENKINS	20
4.1 Alur Proses CI/CD di Jenkins	20
4.2 Membuat Pipeline Job.....	21
4.3 Konfigurasi Source Code Management (Git)	21
4.4 Penulisan Jenkinsfile (Pipeline as Code)	22
4.5 Tahapan Build, Test, dan Deploy	23
4.5.1 Stage: Checkout	23
4.5.2 Stage: Build.....	23
4.5.3 Stage: Test.....	23
4.5.4 Stage: Deploy.....	24
BAB V PEMBAHASAN DAN ANALISIS	25
5.1 Keunggulan Jenkins dalam Ekosistem CI/CD.....	25
5.2 Tantangan Implementasi Jenkins di Windows	25

5.3 Perbandingan Jenkins dengan Tools CI/CD Lainnya	26
5.4 Praktik Terbaik Implementasi Jenkins di Windows	26
BAB VI KESIMPULAN DAN SARAN	28
6.1 Kesimpulan	28
6.2 Saran	29

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan industri perangkat lunak yang pesat menuntut tim pengembang untuk dapat menghasilkan produk berkualitas tinggi dalam waktu yang lebih singkat. Pendekatan pengembangan tradisional seringkali menghasilkan siklus rilis yang panjang, rendahnya kualitas kode, dan seringnya terjadi konflik antar tim pengembang. Untuk mengatasi permasalahan tersebut, metodologi DevOps hadir sebagai solusi yang mengintegrasikan proses pengembangan (Development) dan operasional (Operations) secara sinergis.

Continuous Integration (CI) dan Continuous Deployment (CD) merupakan komponen fundamental dari DevOps yang memungkinkan otomatisasi proses build, testing, dan deployment perangkat lunak. Menurut Kusumadewi dan Adrian (2023), implementasi CI/CD menggunakan Jenkins terbukti mampu meningkatkan performa proses deployment dan mengurangi waktu integrasi secara signifikan. Dengan menggunakan pipeline otomatis, setiap perubahan kode yang di-commit oleh developer dapat segera divalidasi melalui serangkaian proses otomatis tanpa intervensi manual.

Jenkins merupakan salah satu tools CI/CD open-source yang paling banyak digunakan di seluruh dunia. Dengan dukungan lebih dari 1.800 plugin, Jenkins mampu berintegrasi dengan berbagai tools pengembangan perangkat lunak, mulai dari version control system (Git, SVN), build tools (Maven, Gradle), hingga platform cloud (AWS, Azure, GCP). Meskipun Jenkins populer digunakan pada sistem Linux/Unix, implementasinya pada sistem operasi Windows juga sangat relevan, mengingat banyaknya organisasi yang menggunakan Windows sebagai platform utama dalam lingkungan pengembangan mereka.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam makalah ini adalah:

1. Apa yang dimaksud dengan Jenkins dan bagaimana arsitekturnya dalam konteks CI/CD?
2. Bagaimana tahapan instalasi dan konfigurasi Jenkins pada sistem operasi Windows?

3. Bagaimana cara mengimplementasikan pipeline CI/CD menggunakan Jenkins di Windows?
4. Apa saja tahapan yang terlibat dalam proses CI/CD menggunakan Jenkins?

1.3 Tujuan Penulisan

Adapun tujuan penulisan makalah ini adalah:

1. Menjelaskan konsep dasar dan arsitektur Jenkins sebagai automation server CI/CD.
2. Mendeskripsikan tahapan instalasi Jenkins secara lengkap pada sistem operasi Windows.
3. Menguraikan langkah-langkah implementasi pipeline CI/CD berbasis Windows menggunakan Jenkins.
4. Memberikan pemahaman komprehensif mengenai setiap tahapan dalam proses CI/CD Jenkins.

1.4 Manfaat Penulisan

Makalah ini diharapkan dapat memberikan manfaat bagi pembaca, khususnya para pengembang perangkat lunak dan mahasiswa ilmu komputer, dalam memahami konsep dan implementasi praktis Jenkins sebagai tools CI/CD pada lingkungan Windows. Selain itu, makalah ini juga dapat menjadi referensi awal bagi organisasi yang ingin mengadopsi praktik DevOps dalam proses pengembangan perangkat lunak mereka.

BAB II

LANDASAN TEORI

2.1 DevOps

DevOps merupakan singkatan dari Development (pengembangan) dan Operations (operasional), yang mencerminkan pendekatan budaya dan praktik yang bertujuan menyatukan tim pengembang perangkat lunak dengan tim operasional IT. Menurut Narang, Mittal, dan Narang (2023), DevOps adalah metodologi yang memfasilitasi kolaborasi antara pengembang dan tim operasi untuk mengintegrasikan individu, proses, dan teknologi secara erat demi menghasilkan pengiriman perangkat lunak yang otomatis, gesit, skalabel, dan efisien dari segi biaya.

Dalam praktiknya, DevOps mengandalkan serangkaian prinsip dan praktik utama, antara lain: Continuous Integration (CI), Continuous Delivery/Deployment (CD), Infrastructure as Code (IaC), monitoring berkelanjutan, dan kolaborasi lintas tim. Implementasi DevOps yang baik terbukti mampu memperpendek siklus rilis perangkat lunak, meningkatkan frekuensi deployment, dan menurunkan tingkat kegagalan perubahan (change failure rate) secara dramatis.

2.2 Continuous Integration dan Continuous Deployment (CI/CD)

Continuous Integration (CI) adalah praktik pengembangan perangkat lunak di mana setiap anggota tim secara rutin mengintegrasikan perubahan kode ke dalam repository bersama, dengan frekuensi minimal sekali sehari. Setiap integrasi kemudian diverifikasi secara otomatis melalui proses build dan serangkaian pengujian otomatis, sehingga kesalahan (bug) dapat dideteksi lebih awal dalam siklus pengembangan.

Continuous Deployment (CD) merupakan kelanjutan dari CI, di mana setiap perubahan kode yang berhasil melewati seluruh tahapan pengujian secara otomatis dideploy ke lingkungan produksi tanpa memerlukan intervensi manual. Menurut penelitian yang diterbitkan oleh MDPI Sensors (2024), CI/CD merupakan komponen penting dalam ekosistem DevOps yang memungkinkan pengembang untuk sering melakukan commit perubahan kode, yang kemudian secara otomatis diintegrasikan dan diuji untuk meningkatkan keandalan serta efisiensi proses deployment.

Tabel 1. Perbandingan Continuous Integration (CI) dan Continuous Deployment (CD)

Aspek	Continuous Integration (CI)	Continuous Deployment (CD)
Definisi	Proses integrasi kode secara berkala ke repository bersama	Proses deployment otomatis ke lingkungan produksi
Tujuan	Mendeteksi bug lebih awal dalam siklus pengembangan	Mempercepat pengiriman fitur ke pengguna akhir
Frekuensi	Setiap kali ada commit kode (bisa beberapa kali sehari)	Setiap kali build CI berhasil melewati semua test
Output	Laporan build dan hasil pengujian otomatis	Aplikasi yang berjalan di lingkungan produksi
Tools Utama	Jenkins, GitHub Actions, GitLab CI, CircleCI	Jenkins, Spinnaker, ArgoCD, AWS CodeDeploy
Intervensi Manual	Tidak diperlukan (fully automated)	Tidak diperlukan untuk Continuous Deployment; diperlukan untuk Continuous Delivery

2.3 Jenkins sebagai Automation Server

Jenkins adalah automation server open-source yang ditulis dalam bahasa Java, dirancang khusus untuk memfasilitasi proses Continuous Integration dan Continuous Delivery/Deployment. Jenkins pertama kali dikembangkan oleh Kohsuke Kawaguchi pada tahun 2004 dengan nama Hudson, kemudian berganti nama menjadi Jenkins pada tahun 2011 setelah terjadi perpecahan dengan Oracle. Saat ini, Jenkins dikelola oleh komunitas open-source yang besar dan aktif di bawah naungan Linux Foundation's Continuous Delivery Foundation (CDF).

Sebagai automation server, Jenkins bertugas mengorkestrasikan seluruh pipeline CI/CD dengan cara mengotomatisasi proses build, menjalankan pengujian (unit test, integration test), serta memfasilitasi proses deployment ke berbagai lingkungan (development, staging, production). Penelitian oleh Abdel Mowad et al. (2022) menunjukkan bahwa penggunaan CI/CD seperti Jenkins secara signifikan mampu mempersempit kesenjangan antara tim developer dan tim operasi, yang merupakan inti dari filosofi DevOps.

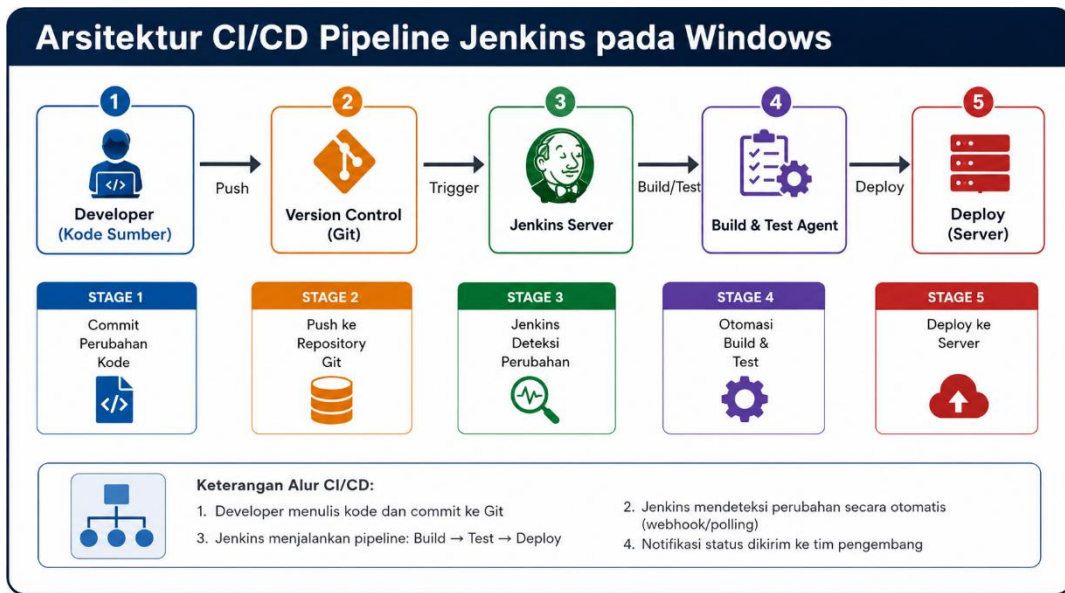
Tabel 2. Fitur-Fitur Utama Jenkins

Fitur	Keterangan
Open Source	Tersedia secara gratis dengan lisensi MIT, komunitas aktif dengan ribuan kontributor global.
Plugin Ecosystem	Lebih dari 1.800 plugin tersedia untuk integrasi dengan tools pengembangan populer (Git, Maven, Docker, AWS, Kubernetes, dll.).
Pipeline as Code	Mendukung penulisan definisi pipeline dalam file Jenkinsfile menggunakan Groovy DSL (Declarative & Scripted Pipeline).
Multi-Platform	Dapat berjalan di Windows, Linux, macOS, dan sebagai container Docker.
Distributed Builds	Mendukung arsitektur master-agent untuk mendistribusikan beban build ke beberapa mesin sekaligus.
Notifikasi Otomatis	Mampu mengirimkan notifikasi hasil build via email, Slack, Microsoft Teams, dan platform komunikasi lainnya.
Antarmuka Web	Menyediakan dashboard berbasis web yang intuitif untuk manajemen job, monitoring build, dan konfigurasi sistem.

2.4 Arsitektur Jenkins

Jenkins menggunakan arsitektur Master-Agent (sebelumnya dikenal sebagai Master-Slave). Dalam arsitektur ini, Jenkins Master bertanggung jawab untuk menjadwalkan build job, mendistribusikan pekerjaan ke agent, memantau status agent, serta menyajikan hasil build melalui antarmuka web. Sementara itu, Jenkins Agent (Worker Node) adalah mesin yang secara aktif menjalankan proses build sesuai instruksi dari Master.

Arsitektur ini memungkinkan Jenkins untuk menskalakan kapasitas build dengan mudah, karena beban pekerjaan dapat didistribusikan ke banyak agent yang berjalan di mesin yang berbeda, bahkan di sistem operasi yang berbeda secara bersamaan. Komunikasi antara Master dan Agent dilakukan melalui protokol Jenkins Remoting menggunakan JNLP atau SSH.



Gambar 1. Arsitektur CI/CD Pipeline Jenkins

BAB III

INSTALASI JENKINS DI WINDOWS

3.1 Prasyarat Sistem

Sebelum melakukan instalasi Jenkins di Windows, terdapat beberapa prasyarat yang harus dipenuhi untuk memastikan Jenkins dapat berjalan dengan optimal. Prasyarat ini meliputi spesifikasi perangkat keras minimum, perangkat lunak pendukung, serta konfigurasi sistem yang diperlukan.

Tabel 3. Prasyarat Sistem untuk Instalasi Jenkins di Windows

Komponen	Minimum	Rekomendasi
Sistem Operasi	Windows 10 (64-bit)	Windows 10/11 Pro atau Windows Server 2019/2022
Prosesor	Dual-core CPU, 1 GHz	Quad-core CPU, 2 GHz atau lebih
RAM	256 MB (hanya Jenkins)	4 GB atau lebih (termasuk aplikasi lain)
Ruang Disk	1 GB (instalasi Jenkins)	10 GB atau lebih (termasuk workspace dan artifact)
Java (JDK)	JDK 11 (sudah tidak didukung sepenuhnya)	JDK 17 atau JDK 21 (LTS versi terbaru)
Browser	Chrome, Firefox, Edge (versi terbaru)	Google Chrome atau Mozilla Firefox terbaru
Koneksi Jaringan	Akses internet untuk unduh plugin	Koneksi stabil dengan bandwidth memadai
Port	Port 8080 (default HTTP)	Port 8080 atau port kustom yang tidak digunakan aplikasi lain

3.2 Instalasi Java Development Kit (JDK)

Jenkins memerlukan Java Runtime Environment (JRE) atau Java Development Kit (JDK) untuk dapat berjalan. Versi JDK yang direkomendasikan adalah JDK 21 (Long-Term Support). Berikut ini adalah langkah-langkah instalasi JDK pada Windows:

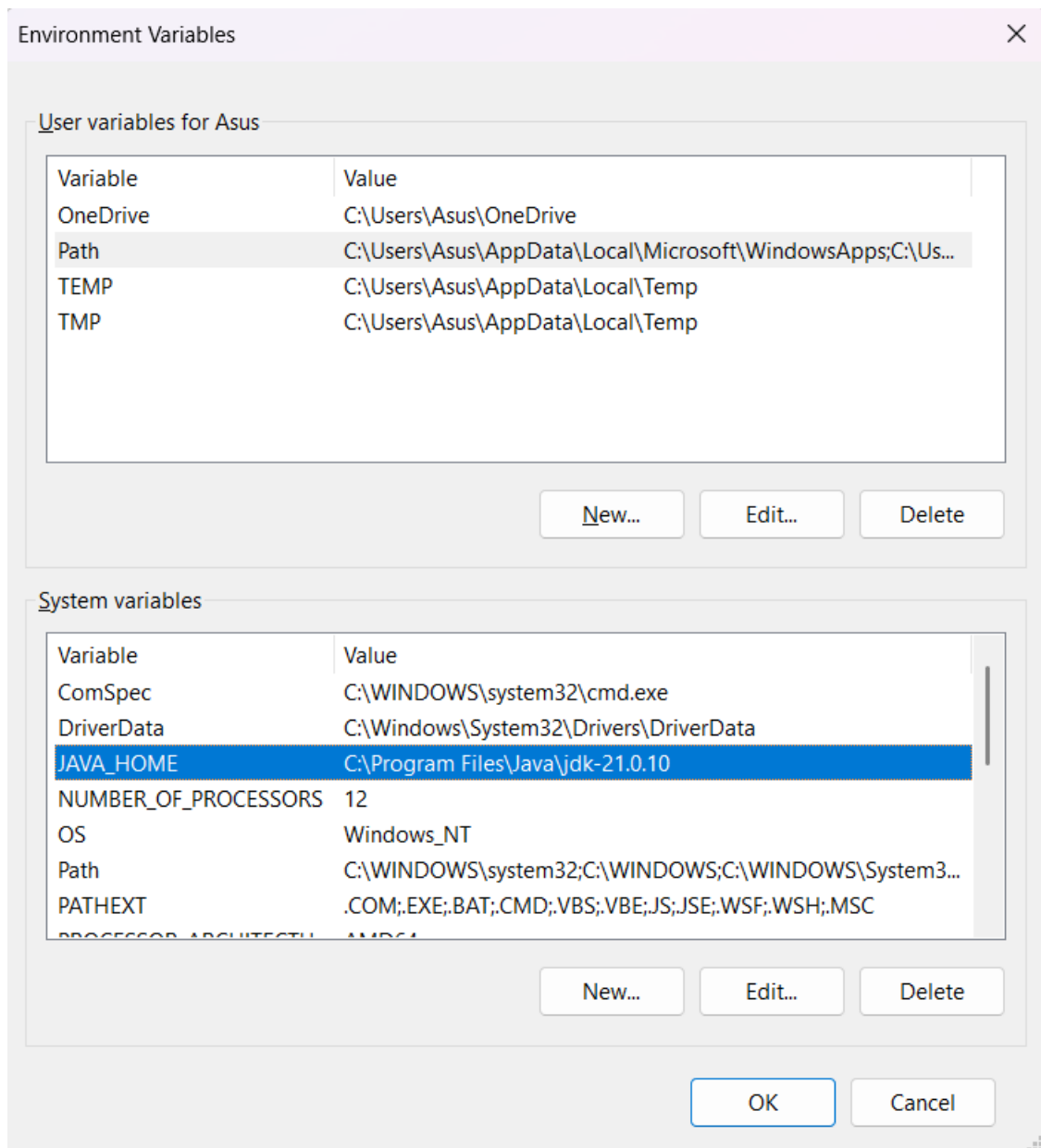
1. Buka browser dan kunjungi laman resmi OpenJDK di <https://adoptium.net> atau situs Oracle di <https://www.oracle.com/java/technologies/downloads/> untuk mengunduh JDK 21.
2. Pilih versi Windows x64 Installer (.msi) dan unduh file installer tersebut.

3. Jalankan file installer (.msi) yang telah diunduh dengan hak akses Administrator (klik kanan > Run as Administrator).



Gambar 2. Tampilan Instalasi Java

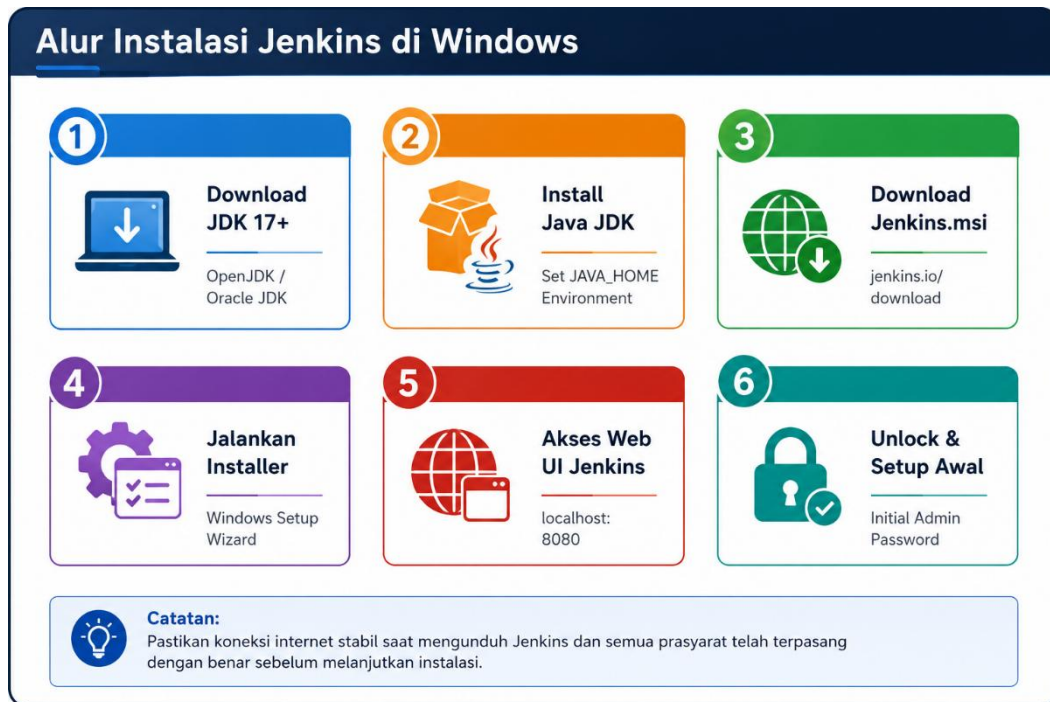
4. Ikuti wizard instalasi dan pastikan opsi 'Set JAVA_HOME variable' dan 'JavaSoft registry keys' dicentang agar variabel lingkungan terkonfigurasi secara otomatis.
5. Setelah instalasi selesai, verifikasi instalasi dengan membuka Command Prompt (cmd) dan ketikkan perintah: `java -version`. Jika instalasi berhasil, sistem akan menampilkan informasi versi Java yang terpasang.
6. Apabila variabel lingkungan JAVA_HOME belum terkonfigurasi otomatis, lakukan konfigurasi manual melalui: Control Panel > System > Advanced System Settings > Environment Variables. Tambahkan JAVA_HOME dengan nilai path direktori JDK (misalnya: C:\Program Files\Java\jdk-21).



Gambar 3. Contoh Pengaturan JAVA_HOME di Environment Variables

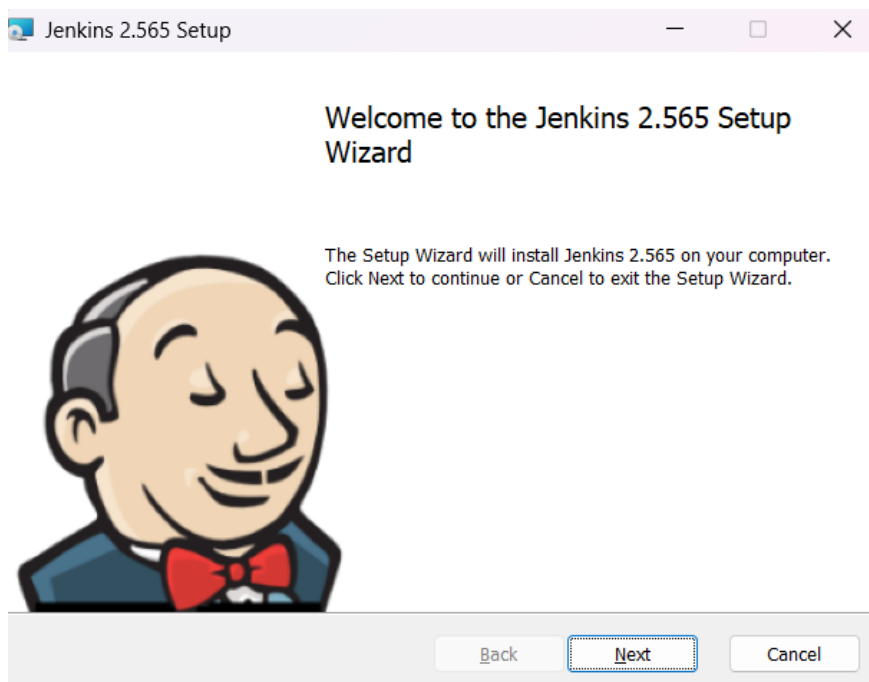
3.3 Instalasi Jenkins melalui Windows Installer

Setelah JDK berhasil dipasang, langkah selanjutnya adalah mengunduh dan memasang Jenkins. Jenkins menyediakan installer khusus untuk Windows dalam format .msi yang memudahkan proses instalasi. Berikut tahapan instalasinya:



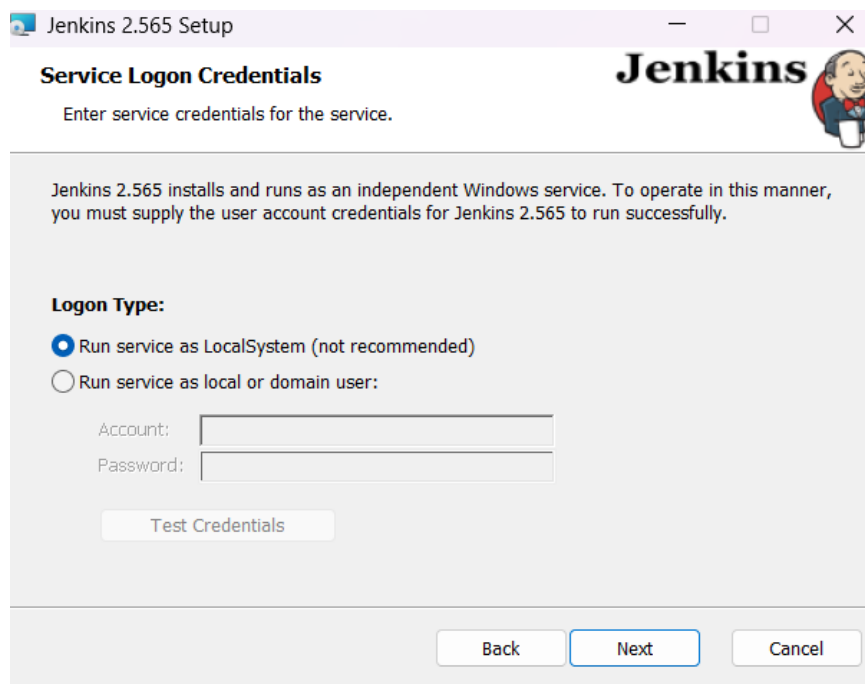
Gambar 4. Alur Instalasi Jenkins di Windows

1. Kunjungi situs resmi Jenkins di <https://www.jenkins.io/download/> dan unduh paket installer untuk Windows (file berekstensi .msi) dari kategori 'Long-term Support (LTS)'. Versi LTS direkomendasikan untuk lingkungan produksi karena lebih stabil.
2. Setelah file .msi selesai diunduh, klik dua kali untuk memulai proses instalasi. Pastikan Anda menjalankannya sebagai Administrator.



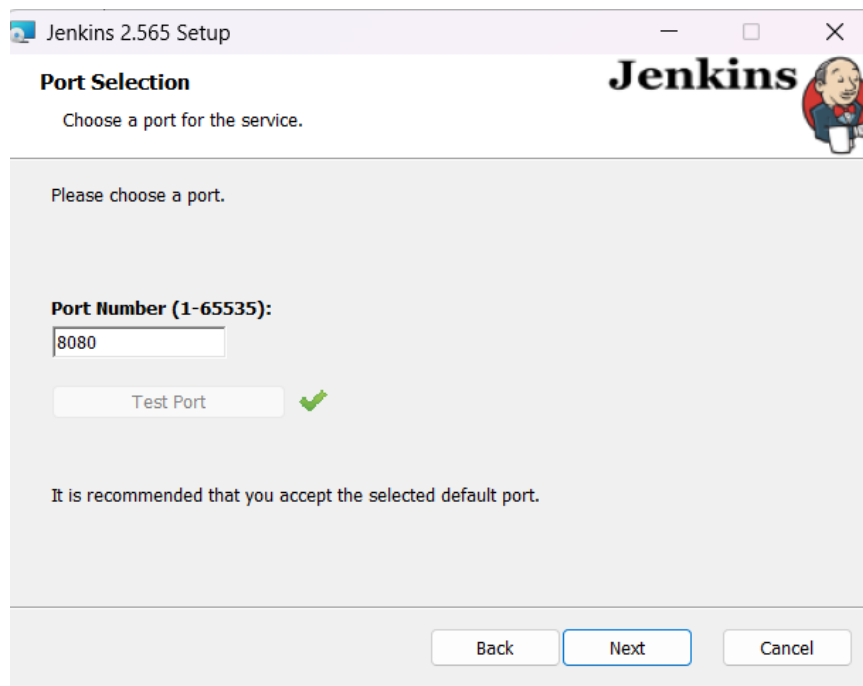
Gambar 6. Tampilan Awal Setup Jenkins

3. Setup Wizard akan terbuka. Klik 'Next' untuk melanjutkan. Pada tahap pemilihan direktori instalasi, Anda dapat membiarkan direktori default (C:\Program Files\Jenkins) atau mengubahnya sesuai kebutuhan.
4. Pada tahap 'Service Logon Credentials', pilih 'Run service as LocalSystem (not recommended)' untuk kemudahan awal, atau konfigurasi akun domain untuk lingkungan enterprise.



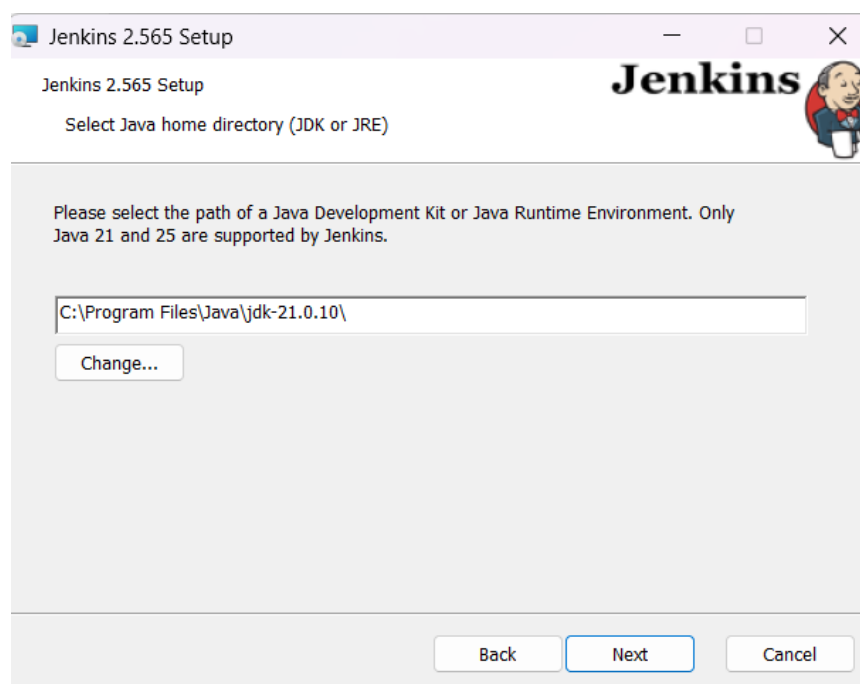
Gambar 7. Menentukan Logon Credentials

5. Selanjutnya, tentukan port yang akan digunakan Jenkins. Port default adalah 8080. Pastikan port tersebut tidak digunakan oleh aplikasi lain. Klik 'Test Port' untuk memverifikasi ketersediaan port tersebut.



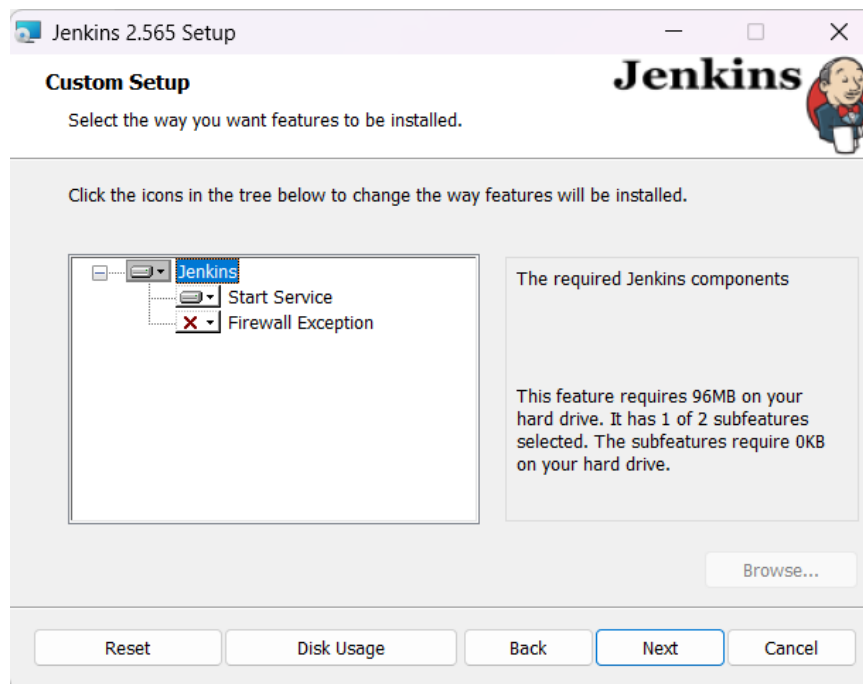
Gambar 8. Menentukan Port

6. Tentukan lokasi instalasi Java dengan mengklik 'Browse' dan arahkan ke direktori JDK yang telah dipasang sebelumnya.



Gambar 9. Memasukkan Lokasi Java

7. Pada halaman 'Custom Setup', pilih fitur-fitur yang ingin dipasang. Untuk sebagian besar kasus, konfigurasi default sudah mencukupi. Klik 'Next' dan kemudian 'Install' untuk memulai proses instalasi.



Gambar 10. Custom Setup

8. Tunggu hingga proses instalasi selesai. Windows mungkin akan meminta izin untuk membuat firewall rule agar Jenkins dapat diakses melalui jaringan. Klik 'Allow Access' untuk mengizinkannya.
9. Setelah instalasi selesai, klik 'Finish'. Layanan Jenkins (Jenkins service) akan berjalan secara otomatis sebagai Windows Service dengan startup type Automatic.

Setelah instalasi selesai, Jenkins dapat diakses melalui browser dengan membuka alamat <http://localhost:8080> atau [http://\[IP-Address\]:8080](http://[IP-Address]:8080) jika diakses dari mesin lain dalam jaringan yang sama.

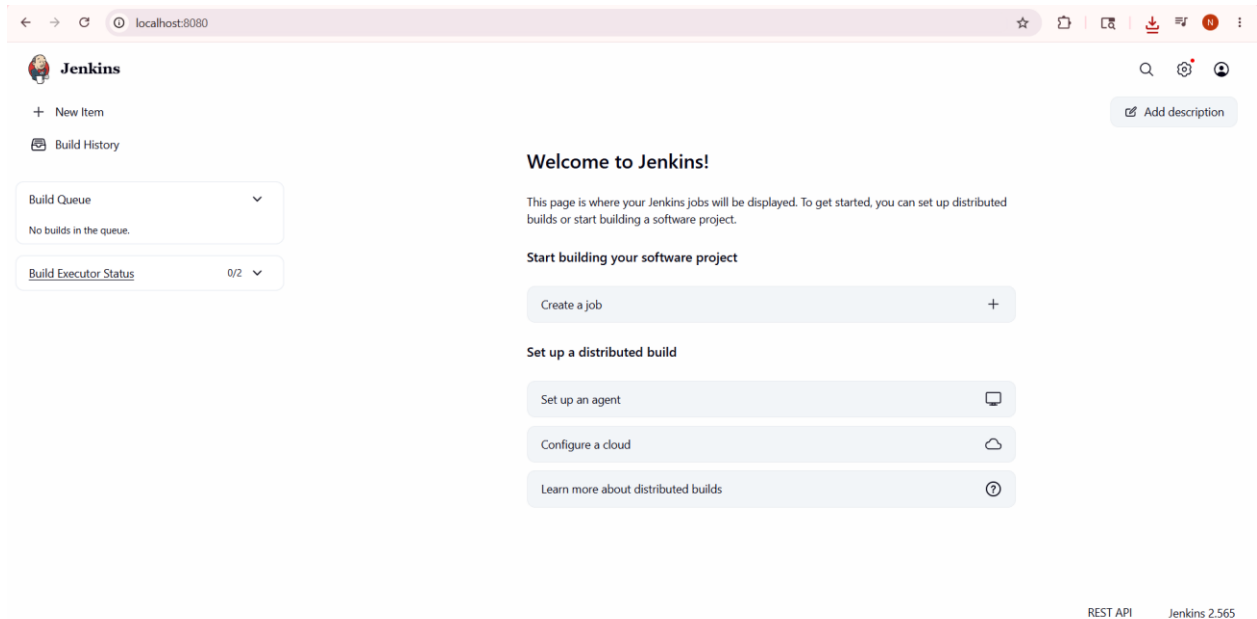
3.4 Konfigurasi Awal Jenkins

Pada saat pertama kali diakses, Jenkins akan meminta sejumlah konfigurasi awal yang perlu diselesaikan melalui Setup Wizard berbasis web. Tahapan konfigurasi awal ini sangat penting untuk memastikan keamanan dan fungsionalitas Jenkins.

3.4.1 Membuka Kunci Jenkins (Unlock Jenkins)

Saat pertama kali mengakses Jenkins melalui browser di <http://localhost:8080>, sistem akan menampilkan halaman 'Unlock Jenkins'. Halaman ini meminta pengguna untuk memasukkan Initial Administrator Password yang secara otomatis dibuat selama proses instalasi. Password ini dapat ditemukan pada file yang berlokasi di: `C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword`.

Buka file tersebut menggunakan Notepad atau editor teks lainnya, salin isinya, dan tempelkan ke kolom yang tersedia pada halaman Unlock Jenkins, lalu klik 'Continue'.



Gambar 11. Tampilan Dashboard Jenkins setelah Konfigurasi Awal

3.4.2 Pemilihan Plugin Awal

Setelah Jenkins terbuka, sistem akan menampilkan halaman 'Customize Jenkins' dengan dua pilihan: 'Install suggested plugins' dan 'Select plugins to install'. Untuk pengguna baru, disarankan untuk memilih 'Install suggested plugins'. Jenkins akan secara otomatis mengunduh dan memasang koleksi plugin yang paling umum digunakan, termasuk plugin untuk Git, Pipeline, Maven, Gradle, dan berbagai plugin lainnya.

Proses pengunduhan dan pemasangan plugin memerlukan koneksi internet dan mungkin memakan waktu beberapa menit tergantung pada kecepatan koneksi. Setelah selesai, Jenkins akan menampilkan halaman pembuatan akun admin pertama.

3.4.3 Pembuatan Akun Administrator

Pada halaman 'Create First Admin User', isi formulir dengan nama pengguna (username), kata sandi (password), nama lengkap, dan alamat email untuk akun administrator Jenkins. Klik 'Save and Continue'. Pada halaman berikutnya, konfigurasi URL Jenkins instance. Biarkan URL default (<http://localhost:8080/>) atau sesuaikan dengan URL server Jenkins yang sebenarnya jika akan diakses dari jaringan lain. Klik 'Save and Finish', lalu 'Start using Jenkins' untuk mengakses dashboard Jenkins.

3.5 Manajemen Plugin Jenkins

Plugin adalah komponen yang memperluas fungsionalitas Jenkins. Pengelolaan plugin dilakukan melalui menu Manage Jenkins > Manage Plugins. Terdapat empat tab utama dalam halaman Plugin Manager, yaitu: Updates (plugin yang perlu diperbarui), Available (plugin baru yang dapat dipasang), Installed (plugin yang sudah terpasang), dan Advanced (konfigurasi sumber plugin).

Beberapa plugin esensial yang perlu dipasang untuk implementasi CI/CD yang lengkap antara lain: Git Plugin (integrasi dengan repository Git), Pipeline Plugin (mendukung pipeline as code), Maven Integration Plugin (untuk proyek berbasis Maven), Credentials Plugin (manajemen kredensial yang aman), Blue Ocean (antarmuka visual pipeline yang modern), dan Email Extension Plugin (notifikasi email).

Tabel 4. Plugin Jenkins Penting untuk CI/CD di Windows

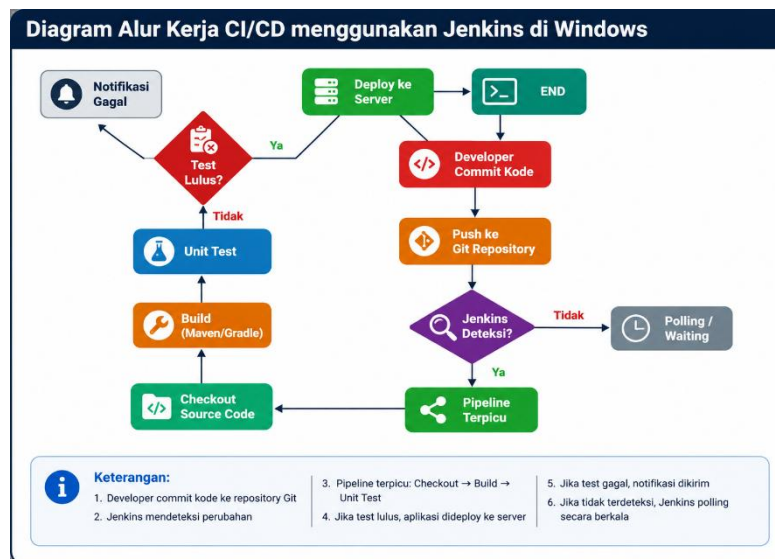
Nama Plugin	Kategori	Fungsi Utama
Git Plugin	SCM	Integrasi Jenkins dengan repository Git (GitHub, GitLab, Bitbucket)
Pipeline Plugin	Pipeline	Mendukung pembuatan dan eksekusi Pipeline as Code (Declarative & Scripted)
Maven Integration	Build Tools	Integrasi dengan Apache Maven untuk proyek Java
Blue Ocean	UI/UX	Antarmuka visual modern untuk visualisasi dan manajemen pipeline CI/CD
Credentials Binding	Security	Mengelola dan menggunakan kredensial (token, password) secara aman dalam pipeline
Workspace Cleanup	Utilities	Membersihkan workspace Jenkins sebelum atau setelah eksekusi build
Email Extension	Notification	Mengirimkan notifikasi email yang dapat dikustomisasi berdasarkan hasil build
Docker Pipeline	Container	Memungkinkan penggunaan Docker container sebagai environment build dalam pipeline

BAB IV

IMPLEMENTASI CI/CD PIPELINE DENGAN JENKINS

4.1 Alur Proses CI/CD di Jenkins

Proses CI/CD di Jenkins mengikuti alur kerja yang sistematis dan terotomasi. Seluruh proses dimulai dari developer yang melakukan commit dan push perubahan kode ke repository Git, hingga aplikasi berhasil dideploy ke server tujuan. Pemahaman yang baik terhadap alur ini sangat penting untuk merancang pipeline yang efektif dan efisien.



Gambar 12. Diagram Alur Kerja CI/CD menggunakan Jenkins di Windows

Berdasarkan diagram alur di atas, proses CI/CD di Jenkins dapat dibagi menjadi beberapa fase utama:

1. Fase Source Code Management: Developer menulis kode, melakukan commit, dan push ke repository Git (GitHub/GitLab/Bitbucket). Setiap perubahan kode menjadi pemicu awal pipeline.
2. Fase Trigger: Jenkins mendeteksi perubahan melalui dua mekanisme, yaitu Webhook (notifikasi push dari Git secara real-time) atau SCM Polling (Jenkins memeriksa repository secara berkala sesuai interval yang dikonfigurasi).
3. Fase Build: Jenkins men-checkout kode terbaru dan menjalankan proses kompilasi menggunakan build tools yang sesuai (misalnya Maven untuk Java, MSBuild untuk .NET, npm untuk Node.js).
4. Fase Test: Jenkins menjalankan serangkaian automated test, mulai dari unit test, integration test, hingga code quality analysis menggunakan tools seperti SonarQube.

5. Fase Deploy: Jika semua tahapan test berhasil dilewati, Jenkins secara otomatis melakukan deployment aplikasi ke server target (development, staging, atau production).
6. Fase Notifikasi: Jenkins mengirimkan notifikasi hasil build (sukses/gagal) kepada tim pengembang melalui email, Slack, atau platform komunikasi lainnya.

4.2 Membuat Pipeline Job

Terdapat dua jenis pipeline job utama yang dapat dibuat di Jenkins, yaitu Freestyle Project (konfigurasi berbasis GUI) dan Pipeline (konfigurasi berbasis kode/Jenkinsfile). Untuk implementasi CI/CD yang lebih fleksibel dan dapat direproduksi, Pipeline Job dengan Jenkinsfile adalah pilihan yang direkomendasikan.

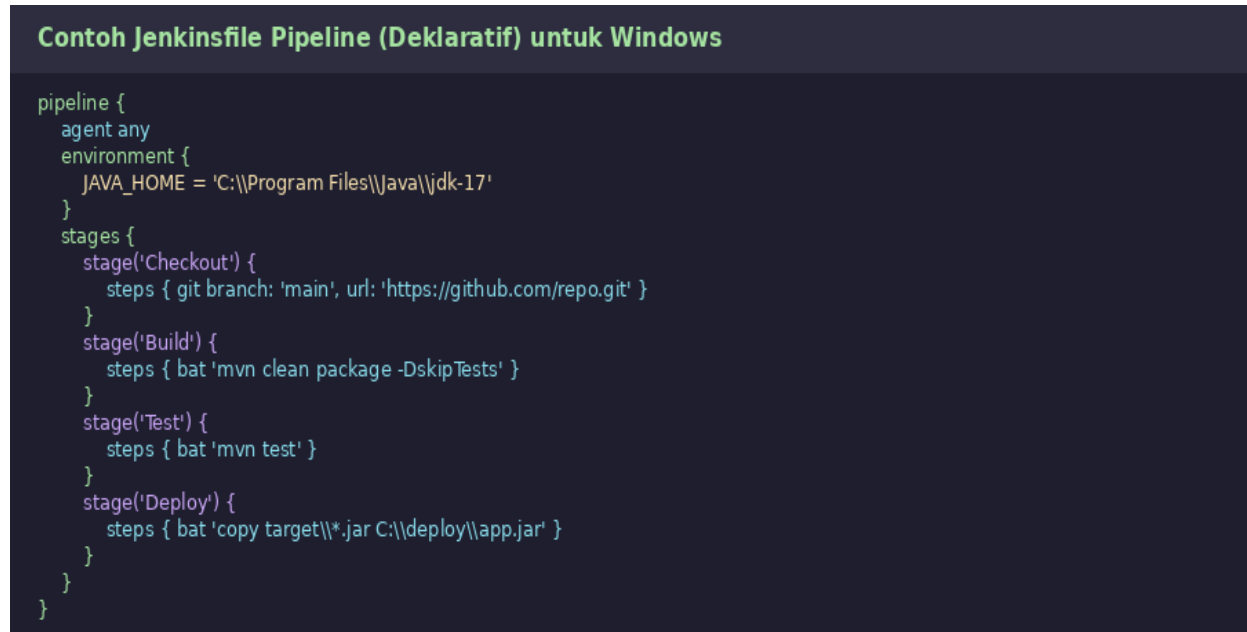
Langkah-langkah membuat Pipeline Job baru di Jenkins adalah sebagai berikut: Buka dashboard Jenkins dan klik 'New Item'. Masukkan nama item (misalnya: WebApp-CI-Pipeline), pilih tipe 'Pipeline', lalu klik 'OK'. Pada halaman konfigurasi job, isi deskripsi proyek di bagian General. Pada bagian Build Triggers, pilih mekanisme trigger yang diinginkan, misalnya 'GitHub hook trigger for GITScm polling' untuk trigger berbasis webhook, atau 'Poll SCM' dengan jadwal cron expression (misalnya: H/5 * * * * untuk polling setiap 5 menit). Pada bagian Pipeline, pilih antara 'Pipeline script' (menulis langsung di UI) atau 'Pipeline script from SCM' (membaca Jenkinsfile dari repository Git).

4.3 Konfigurasi Source Code Management (Git)

Konfigurasi integrasi Jenkins dengan repository Git dilakukan melalui beberapa langkah. Pada bagian Pipeline Definition, pilih 'Pipeline script from SCM', kemudian pilih 'Git' sebagai SCM. Isikan URL repository Git (contoh: <https://github.com/username/repository.git>). Jika repository bersifat privat, tambahkan kredensial melalui menu 'Add' di bagian Credentials. Pilih jenis kredensial yang sesuai, misalnya 'Username with password' untuk autentikasi menggunakan akun GitHub/GitLab, atau 'Secret text' untuk Personal Access Token (PAT). Tentukan branch yang akan dimonitor (misalnya: */main atau */master). Pada kolom 'Script Path', masukkan path relatif ke file Jenkinsfile dalam repository (default: Jenkinsfile). Klik 'Save' untuk menyimpan konfigurasi.

4.4 Penulisan Jenkinsfile (Pipeline as Code)

Jenkinsfile adalah file teks yang mendefinisikan seluruh pipeline CI/CD menggunakan Groovy-based DSL (Domain Specific Language). File ini disimpan langsung di dalam repository proyek, sehingga konfigurasi pipeline dapat diversi dan dikelola bersama source code. Terdapat dua sintaks Jenkinsfile, yaitu Declarative Pipeline (lebih terstruktur dan mudah dibaca) dan Scripted Pipeline (lebih fleksibel namun lebih kompleks). Berikut ini adalah contoh Jenkinsfile Deklaratif untuk proyek Java berbasis Maven yang berjalan di Windows:



```
pipeline {
  agent any
  environment {
    JAVA_HOME = 'C:\\Program Files\\Java\\jdk-17'
  }
  stages {
    stage('Checkout') {
      steps { git branch: 'main', url: 'https://github.com/repo.git' }
    }
    stage('Build') {
      steps { bat 'mvn clean package -DskipTests' }
    }
    stage('Test') {
      steps { bat 'mvn test' }
    }
    stage('Deploy') {
      steps { bat 'copy target\\*.jar C:\\deploy\\app.jar' }
    }
  }
}
```

Gambar 13. Contoh Kode Jenkinsfile Pipeline Deklaratif untuk Windows

Pada contoh Jenkinsfile di atas, perlu diperhatikan penggunaan perintah bat (batch) sebagai pengganti sh (shell) yang umum digunakan di Linux. Perintah bat digunakan untuk menjalankan perintah Windows Command Prompt (CMD) atau PowerShell. Selain itu, path direktori menggunakan format Windows dengan pemisah backslash yang harus di-escape (\\) dalam string Groovy.

4.5 Tahapan Build, Test, dan Deploy

Setiap stage dalam Jenkinsfile merepresentasikan satu tahapan dalam pipeline CI/CD. Berikut ini adalah penjelasan detail masing-masing tahapan:

4.5.1 Stage: Checkout

Tahapan Checkout bertugas mengunduh source code terbaru dari repository Git ke workspace Jenkins. Jenkins secara otomatis melakukan git clone atau git pull ke direktori workspace yang berlokasi di `C:\ProgramData\Jenkins\.jenkins\workspace\[nama-job]\` pada instalasi Windows. Konfigurasi checkout dapat dilakukan menggunakan step `git()` bawaan Jenkins atau dengan menggunakan `checkout(scm)` yang membaca konfigurasi SCM dari job configuration.

4.5.2 Stage: Build

Tahapan Build bertugas mengkompilasi source code dan menghasilkan artifact yang dapat dideploy. Pada proyek Java berbasis Maven, perintah yang dijalankan adalah `mvn clean package -DskipTests`. Opsi `-DskipTests` digunakan untuk melewati eksekusi test pada tahap build, karena pengujian akan dilakukan secara terpisah pada tahap berikutnya. Untuk proyek .NET di Windows, perintah yang digunakan adalah `msbuild Project.csproj /t:Build /p:Configuration=Release`. Sedangkan untuk proyek Node.js, perintah yang digunakan adalah `npm install` diikuti `npm run build`.

4.5.3 Stage: Test

Tahapan Test menjalankan berbagai jenis pengujian otomatis untuk memastikan kualitas kode sebelum dideploy. Untuk proyek Maven, perintah `mvn test` akan menjalankan seluruh unit test yang tersedia. Jenkins dapat mengumpulkan laporan hasil pengujian dalam format JUnit XML menggunakan step `junit('**/target/surefire-reports/*.xml')`. Laporan ini kemudian divisualisasikan dalam dashboard Jenkins sebagai trend grafik hasil test. Jika ada test yang gagal, Jenkins secara otomatis menghentikan pipeline dan menandai build sebagai 'FAILED'.

4.5.4 Stage: Deploy

Tahapan Deploy bertanggung jawab untuk mendistribusikan artifact hasil build ke lingkungan target. Pada Windows, deployment dapat dilakukan dengan beberapa cara, antara lain: menyalin file JAR/WAR menggunakan perintah xcopy atau robocopy, melakukan deployment ke Windows IIS Server, mengirimkan artifact ke server remote menggunakan WinRM, atau mendeploy ke platform cloud menggunakan AWS CLI, Azure CLI, atau Google Cloud SDK.

Tabel 5. Perbandingan Perintah Jenkins antara Windows dan Linux

Fungsi	Windows (bat/powershell)	Linux/macOS (sh)
Eksekusi Perintah	bat 'perintah'	sh 'perintah'
Build Maven	bat 'mvn clean package'	sh 'mvn clean package'
Salin File	bat 'xcopy /E /I src dest'	sh 'cp -r src dest'
Hapus Direktori	bat 'rmdir /S /Q direktori'	sh 'rm -rf direktori'
Variabel Lingkungan	bat 'echo %JAVA_HOME%'	sh 'echo \$JAVA_HOME'
PowerShell	powershell 'Get-Process'	Tidak ada padanan langsung
Path Separator	Backslash (\) atau Forward Slash (/)	Forward Slash (/) saja

BAB V

PEMBAHASAN DAN ANALISIS

Berdasarkan pembahasan yang telah diuraikan pada bab-bab sebelumnya, terdapat beberapa aspek penting yang perlu dianalisis lebih mendalam terkait implementasi Jenkins untuk CI/CD pada sistem operasi Windows.

5.1 Keunggulan Jenkins dalam Ekosistem CI/CD

Jenkins menunjukkan beberapa keunggulan kompetitif dibandingkan tools CI/CD lainnya, terutama dalam hal fleksibilitas dan ekosistem plugin yang matang. Penelitian yang dilakukan oleh Kusumadewi dan Adrian (2023) yang dipublikasikan dalam MATICS: Jurnal Ilmu Komputer dan Teknologi Informasi menunjukkan bahwa implementasi CI/CD DevOps menggunakan Jenkins menghasilkan peningkatan performa yang signifikan, di mana penggunaan sumber daya CPU dan memori pada pipeline Jenkins lebih efisien dibandingkan dengan eksekusi manual tanpa otomasi.

Selain itu, penelitian yang diterbitkan di jurnal MDPI Sensors (2024) dengan judul 'The Impact of an Automation System Built with Jenkins on the Efficiency of Container-Based System Deployment' mengkonfirmasi bahwa Jenkins mampu secara dramatis meningkatkan efisiensi proses deployment berbasis container. Penelitian tersebut menunjukkan bahwa penggunaan Jenkins sebagai automation server dapat mempersingkat durasi deployment hingga 70% dibandingkan dengan proses manual, serta secara signifikan mengurangi tingkat kesalahan human error.

5.2 Tantangan Implementasi Jenkins di Windows

Meskipun Jenkins mendukung Windows secara penuh, terdapat beberapa tantangan spesifik yang perlu diperhatikan dalam konteks implementasi di lingkungan Windows. Pertama, perbedaan sintaks perintah antara Windows (menggunakan bat atau powershell) dan Linux (menggunakan sh) dapat menjadi sumber kebingungan, terutama bagi tim yang sebelumnya telah terbiasa dengan Jenkinsfile berbasis Linux. Kedua, perbedaan format path (backslash vs forward slash) dan variabel lingkungan (misalnya %VARIABLE% di Windows vs \$VARIABLE di Linux) perlu diantisipasi dalam penulisan Jenkinsfile.

Ketiga, beberapa plugin Jenkins tidak secara resmi mendukung Windows atau memiliki keterbatasan fungsionalitas di platform tersebut. Oleh karena itu, pemilihan plugin harus dilakukan dengan cermat dan diuji terlebih dahulu di lingkungan Windows. Keempat, konfigurasi firewall dan Windows Defender mungkin perlu disesuaikan agar Jenkins dapat berkomunikasi dengan agent dan tools eksternal lainnya.

5.3 Perbandingan Jenkins dengan Tools CI/CD Lainnya

Dalam penelitian yang dipublikasikan oleh Putra et al. (2022) di Jurnal Teknik ITS, dilakukan perbandingan antara berbagai tools CI/CD termasuk Jenkins, GitHub Actions, dan GitLab CI/CD. Hasil penelitian menunjukkan bahwa Jenkins memiliki keunggulan dalam hal fleksibilitas konfigurasi dan dukungan terhadap berbagai environment, meskipun memerlukan lebih banyak overhead konfigurasi awal dibandingkan tools berbasis cloud seperti GitHub Actions. Sementara itu, penelitian perbandingan antara Jenkins dan GitLab CI/CD (Telkom University, 2023) menemukan bahwa Jenkins menunjukkan rata-rata waktu yang lebih cepat dalam menyelesaikan proses build dan deploy, sehingga Jenkins memiliki delivery time yang lebih baik meskipun konfigurasinya lebih kompleks.

5.4 Praktik Terbaik Implementasi Jenkins di Windows

Berdasarkan kajian literatur dan praktik industri, terdapat beberapa praktik terbaik (best practices) yang direkomendasikan dalam implementasi Jenkins di Windows:

1. Gunakan Jenkinsfile tersimpan di repository (Pipeline from SCM) agar pipeline dapat diversi-kontrol bersama source code.
2. Manfaatkan Jenkins Shared Libraries untuk mendefinisikan fungsi pipeline yang dapat digunakan ulang (reusable) di berbagai job.
3. Implementasikan parallel stages untuk menjalankan pengujian secara paralel dan mempercepat proses CI/CD.
4. Gunakan Jenkins Credentials Store untuk menyimpan informasi sensitif seperti API key, token, dan password secara aman.
5. Aktifkan Jenkins Security Realm dan Authorization Strategy untuk mengontrol akses pengguna.
6. Lakukan backup reguler terhadap direktori JENKINS_HOME untuk mencegah kehilangan konfigurasi.

7. Pantau penggunaan sumber daya server Jenkins menggunakan plugin Monitoring atau integrasi dengan Prometheus/Grafana.

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Berdasarkan pembahasan yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Jenkins merupakan tools automation server open-source yang sangat powerful dan fleksibel untuk mengimplementasikan pipeline CI/CD. Dengan dukungan lebih dari 1.800 plugin, Jenkins dapat berintegrasi dengan hampir semua tools dalam ekosistem pengembangan perangkat lunak modern.
2. Proses instalasi Jenkins di Windows berjalan melalui beberapa tahapan yang sistematis, dimulai dari persiapan prasyarat sistem (JDK 17+), unduhan dan eksekusi installer .msi, konfigurasi Windows Service, hingga pengaturan awal melalui antarmuka web di port 8080.
3. Implementasi CI/CD pipeline menggunakan Jenkins di Windows melibatkan pembuatan Jenkinsfile yang menggunakan sintaks khusus Windows (perintah bat/powershell), konfigurasi integrasi dengan Git repository, serta pengaturan trigger otomatis (webhook atau SCM polling).
4. Setiap tahapan dalam pipeline CI/CD (Checkout, Build, Test, Deploy, Notify) memiliki peran yang krusial dan saling bergantung. Kegagalan pada satu tahapan akan menghentikan seluruh pipeline dan memberikan notifikasi kepada tim, sehingga memungkinkan deteksi masalah yang cepat.
5. Berdasarkan kajian literatur dari berbagai jurnal ilmiah terkini, implementasi Jenkins untuk CI/CD terbukti mampu meningkatkan efisiensi proses deployment, mengurangi human error, memperpendek siklus rilis, dan meningkatkan kualitas perangkat lunak secara keseluruhan.

6.2 Saran

Adapun beberapa saran yang dapat diberikan berdasarkan kajian ini adalah:

1. Bagi organisasi yang baru memulai adopsi Jenkins, disarankan untuk memulai dengan pipeline sederhana yang mencakup tahapan dasar (build dan test) sebelum menambahkan kompleksitas seperti multi-environment deployment dan parallel stages.
2. Pertimbangkan penggunaan Jenkins dalam container Docker untuk memudahkan portabilitas dan konsistensi environment, meskipun untuk Windows perlu memperhatikan kompatibilitas Windows Container.
3. Investasi dalam pelatihan tim pengembang mengenai konsep DevOps, CI/CD, dan penulisan Jenkinsfile sangat direkomendasikan untuk memaksimalkan manfaat Jenkins dalam jangka panjang.
4. Untuk lingkungan enterprise dengan banyak proyek, pertimbangkan implementasi Jenkins dengan arsitektur distributed (master-agent) untuk meningkatkan kapasitas dan keandalan sistem CI/CD.
5. Penelitian lebih lanjut diperlukan untuk mengkaji perbandingan performa dan efisiensi antara Jenkins yang berjalan native di Windows dengan Jenkins yang berjalan dalam container Docker di lingkungan Windows, khususnya dalam konteks pipeline CI/CD skala besar.

DAFTAR PUSTAKA

- Abdel Mowad, M., Fawareh, H., & Al Hassan, M. (2022). Effect of Using Continuous Integration (CI) and Continuous Delivery (CD) Deployment in DevOps to Reduce the Gap Between Developer and Operation. *Proceedings of the 2022 International Arab Conference on Information Technology (ACIT)*, 1–8. <https://doi.org/10.1109/ACIT57182.2022.10010135>
- Jaeni, Nurhayati, S. A., & Arif, L. D. (2022). Implementasi Continuous Integration/Continuous Delivery (CI/CD) pada Performance Testing DevOps. *Journal of Information System Management (JOISM)*, 4(1), 62–66. <https://doi.org/10.24076/joism.2022v4i1.706>
- Kusumadewi, R. T., & Adrian, R. (2023). Performance Analysis of DevOps Practice Implementation of CI/CD Using Jenkins. *MATICS: Jurnal Ilmu Komputer dan Teknologi Informasi (Journal of Computer Science and Information Technology)*, 15(2), 90–95. <https://doi.org/10.18860/mat.v15i2.21741>
- Narang, P., Mittal, P., & Narang, V. (2023). Automated Continuous Deployment of Software Projects with Jenkins through DevOps-Based Hybrid Model. *Research Square (Preprint)*. <https://doi.org/10.21203/rs.3.rs-2847312/v1>
- Putra, R. A., Studiawan, H., & Rizky, J. A. (2022). Implementasi Continuous Integration dan Continuous Delivery pada Aplikasi myITS Single Sign On. *Jurnal Teknik ITS*, 11(3), A382–A388. <https://doi.org/10.12962/j23373539.v11i3.97542>
- Setyoko, A. D., & Zahra, A. (2024). Perbandingan Efisiensi Proses CI/CD Multi-Lingkungan melalui Implementasi Paralel dan Berurutan. *MALCOM: Indonesian Journal of Machine Learning and Computer Science*, 4(3), 911–925. <https://doi.org/10.57152/malcom.v4i3.1278>
- Shams, S., Salah, I., & Al-Sarayrah, W. (2024). The Impact of an Automation System Built with Jenkins on the Efficiency of Container-Based System Deployment. *Sensors*, 24(18), 6002. <https://doi.org/10.3390/s24186002>
- Ternate, T., Nurhayati, & Widiyanto, R. (2024). Otomatisasi Proses Deployment dengan Metode CI/CD Menggunakan Jenkins dan Docker pada Web Service i-Lab. *Jurnal Repositor*, 6(4). <https://doi.org/10.22219/repositor.v6i4.34109>